12. Kod Okuma Süper Gücü: Yabancı Bir Kod Tabanını AI ile 10 Dakikada Haritalamak

200.000 Satırlık Monolite İlk Kez Bakan Junior'ın Hayatta Kalma Kılavuzu
Bölüm 2 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 13 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Bir şirkette ilk iş gününüzde 200.000 satırlık devasa bir kod tabanıyla karşılaştığınızda, günlerce kaybolmak yerine AI ile sistemi nasıl 1 saatte çözebilirsiniz?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Yazılım mühendisleri zamanlarının %70'ini kod yazarak değil, mevcut kodu okuyarak ve anlayarak geçirir. - 📌 Bilinmeyen bir projeyi incelerken ilk bakılacak yerler: `package.json`, giriş noktası (`index/main`) ve veritabanı şemalarıdır. - 📌 AI'a 'Tüm projeyi oku' demek yerine; 'Giriş isteği (request) geldiğinde hangi katmanlardan geçiyor?' gibi akış soruları sorun.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Kendinizi yabancı bir metropolde, örneğin Tokyo'da bir sokak ortasında hayal edin. Elinizde harita yok, tabelalardaki yazıları okuyamıyorsunuz ve binalar gökyüzüne uzanıyor.

Geleneksel bir junior mühendis 500 dosyalık bir şirket projesini açtığında tam olarak bu Tokyo sendromunu yaşar: *"Ben neredeyim? Bu dosya nereye bağlanıyor?"*

Akıllı bir gezgin ne yapar? Önce metro haritasını açar, ana hatları (kırmızı, mavi hatlar) öğrenir.

Kod tabanının metro haritası: 1. Giriş kapısı (Main / Entry Point) 2. Veritabanı modelleri (Data Models) 3. API yönlendiricileri (Router / Controllers)

Yapay zeka ise elinizdeki akıllı navigasyon cihazıdır. Doğru soruları sorduğunuzda sizi 10 dakikada şehrin merkezine ulaştırır.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Kod Tabanı Haritalama Hiyerarşisi (Top-Down Yaklaşım):

```
1. MANİFESTO: package.json / pyproject.toml
   └── Teknoloji yığını, bağımlılıklar, scriptler

2. VERİ MODELİ: schema.prisma / models/
   └── Bu sistem neyi saklıyor? (Kullanıcı, Ürün, Fatura)

3. GİRİŞ KAPISI: src/main.ts / app.py
   └── Sunucu nasıl ayağa kalkıyor? Hangi middleware'ler devrede?

4. İŞ MANTIĞI: services/ / use-cases/
   └── Gerçek dünya kuralları nerede işleniyor?
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Kıdemli Yazılım Mimarı Onboarding Rehberi
Projeye yeni katılan bir mühendisim. Projemizde aşağıdaki 3 ana dosya bulunmaktadır:
1. @package.json
2. @schema.prisma
3. @src/server.ts

## GÖREV:
Bu 3 dosyayı inceleyerek bana şu konularda 1 sayfalık yönetici özeti sun:
1. Bu sistem temel olarak ne işe yarar (iş modeli nedir)?
2. Bir HTTP isteği geldiğinde izlenen tipik akış mimarisi nasıldır?
3. Projeyi yerel makinemde ayağa kaldırmak için çalıştırmam gereken komut sırası
nedir?
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
# Kötü Yaklaşım: Nereden başlayacağını bilmeden rastgele dosyaları açıp kaybolmak
# 500 dosya arasında kaybolup 3 gün boyunca hiçbir şey anlayamamak...
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```
# Akıllı Haritalama İş Akışı:
# 1. Proje bağımlılıklarını ve çalıştırma komutlarını incele
cat package.json | grep -A 10 "scripts"

# 2. Veritabanı modellerini haritala
cat prisma/schema.prisma | grep "model "

# 3. Testleri çalıştırarak canlı doğrulama yap
npm run test:unit

# 4. Cursor / AI aracında @codebase ile sistem mimarisi diyagramı iste:
# "Generate a mermaid diagram illustrating the lifecycle of an order creation
request."
```

💡 **Kod Analizi:** Yeni bir projeye girdiğinizde rastgele dosya okumak yerine `scripts` alanını ve veritabanı modellerini incelemek size 10 kat daha hızlı resmin bütününü verir. AI'a Mermaid formatında mimari diyagram çizdirmek akılda kalıcılığı artırır.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** İlk günden tüm kod tabanını baştan sona okumaya çalışıp tükenmek.
  - **Çözüm:** Sadece üzerinde çalışacağınız ilk biletin (ticket) dokunduğu dosyaları izole edin.
- **Tuzak:** Projeyi lokalde çalıştırmadan sadece statik kod okumak.
  - **Çözüm:** İlk gün hedefiniz projeyi yerel makinenizde hatasız derleyip testleri yeşile döndürmek olsun.
- **Tuzak:** Ekip arkadaşlarına sormadan önce AI ile kodun amacını analiz etmemek.
  - **Çözüm:** Senior'a gitmeden önce AI ile temel akışı çıkarın; soru sorarken 'X dosyasını inceledim, Y akışını teyit etmek istiyorum' deyin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 12: Kod Okuma Süper Gücü - Kod Tabanını Haritalamak*

- 🖥️ **Slayt 1:** Yazılımcılık neden %70 kod okuma, %30 kod yazmadır?
- 🖥️ **Slayt 2:** 3 Nokta Kılavuzu: Manifestolar, Veri Modelleri, Giriş Noktaları.
- 🖥️ **Slayt 3:** Mermaid.js ile AI destekli anlık mimari akış diyagramları üretmek.
- 🖥️ **Slayt 4:** Bölüm 2 Özeti: Algoritmik Düşünceden Büyük Resme.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Açık kaynaklı popüler bir repoyu (örneğin Redux Toolkit veya tRPC) açın. Cursor'da `@codebase` kulla-

narak: 'Generate a Mermaid sequence diagram for a basic request' komutunu verin. Çıkan diyagramı Markdown önizlemesinde inceleyerek akışı anlatın.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

GitHub'da yıldızı yüksek bir açık kaynak projesini klonlayın. AI asistanınızı kullanarak 15 dakika içinde projenin ne işe yaradığını ve veri tabanı modellerini özetleyen bir `ARCHITECTURE.md` dosyası çıkartın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Yabancı bir yazılım projesine ilk kez katıldığınızda sistemin ne yaptığını anlamak için en stratejik başlangıç noktası neresidir? - A) Rastgele bir CSS dosyası - B) Proje manifestosu (package.json / pyproject.toml) ve veritabanı şemaları (schema / models) - C) .gitignore dosyası - D) README dosyasındaki telif hakları bölümü > **Doğru Yanıt:** B > **Açıklama:** Manifesto projenin teknoloji yığınını ve araçlarını gösterirken, veritabanı şemaları sistemin işlediği gerçek dünya nesnelerini (User, Order vb.) ortaya koyar.

Soru 2: AI yardımıyla bir kod tabanının akışını görselleştirmek için en yaygın kullanılan markdown tabanlı diyagram dili hangisidir? - A) HTML5 - B) Mermaid.js - C) SASS - D) Dockerfile > **Doğru Yanıt:** B > **Açıklama:** Mermaid.js, doğrudan markdown dosyaları içinde sequence, flowchart ve class diyagramları oluşturmaya yarayan endüstri standardı bir araçtır.